

CURSO POWERSHELL OPENWEBINARS.

1. GUI / CLI

La interfaz de usuario se llama **GUI**

La interfaz de línea de comandos se llama **CLI**

2. LINEA DE COMANDOS

Existe la **CMD** o símbolo de sistema → Para tareas sencillas y simples

PowerShell → Reemplaza a la CMD siendo más potente

3. POWERSHELL

Es una herramienta orientada a automatizar tareas, siendo excepcional para administradores.

Trabaja con objetos

Funciona con scripts.

No es case sensitive

PowerShell ISE → Entorno para ejecutar comandos y scripts

PowerShell → Es la línea de comandos

4. ABRIR POWERSHELL

Se puede abrir haciendo clic derecho en el icono de Windows o si queremos abrirlo en una ruta concreta, hacemos clic derecho en la carpeta y le damos a abrir en powershell.

También podemos abrirlo con Win+R y escribimos powershell.

5. CONCEPTO DE CMDLET

Un cmdlet es una combinación de verbo y nombre separados por un guión (-).

Si por ejemplo usamos, Get-LocalUser → Nos devuelve los usuarios locales del equipo.

6. CONCEPTO DE MÓDULO

Los cmdlet se agrupan en **conjuntos**, denominados **módulos**.

Con **Get-Module**, vemos todos los módulos que hay en el sistema.

Get-Command -module <Nombre del Módulo> → Vemos los comandos que hay en el módulo.

Para ver la lista de módulos disponibles → **Get-Module -ListAvailable**

Si queremos importar un módulo → **Import-Module <nombre_módulo>**

Si queremos eliminar un módulo → **Remove-Module <nombre_módulo>**

7. ATAJS Y ALIAS

Son **mecanismos** utilizados para escribir más rápido en el powerShell.

Get-History → Nos devuelve el historial de comandos utilizados → **El atajo es h**

Invoke-History <n> → Realiza el comando ubicado en la file 'n' → **El atajo es r <n>**

Con '**ctrl+r**' entramos en un menú de búsqueda.

Un **alias** es un **apodo** de un comando.

Clear-Host tiene dos alias → Clear o Cls

Si yo quiero buscar un alias → **Get-Alias -Definition <comando>**

ni <nombre_fichero> → Crea un fichero nuevo

md <nombre_directorio> → Crea un directorio nuevo

8. GESTIÓN DE ARCHIVOS Y CARPETAS

Get-Location → **pwd**

Set-Location → **cd**

Get-ChildItem → **ls**

Copy-item → **cp**

Get-Content → **cat**

Cuando utilizamos **Get-ChildItem**, la primera columna no muestra una información de gran utilidad.

- **d** → directorios
- **r** → solo lectura
- **a** → archivo
- **s** → archivos de sistemas
- **h** → archivos ocultos

Para hacer una búsqueda de unos determinados archivos → **Get-ChildItem C:\ -Attributes hidden**

9. TUBERIAS Y REDIRECCIONAMIENTO

Tubería → Conecta la salida de un comando con la entrada de otro con el carácter '|'.

Redireccionamiento → Permite mandar la información a un lugar diferente de la pantalla con los caracteres '>' y '>>'.
> → Redirige la salida a un archivo.
>> → Redirige la salida a un archivo, creando el archivo si no existe.

Get-NetTCPConnection | Where-Object {\$_.State -eq 'Established'} | ft -a → Nos muestra las conexiones de nuestro sistema que han sido establecidas

10. INICIACIÓN DE LOS SCRIPTS

Es necesario que el archivo tenga extensión .ps1 para considerarse un script.

Tiene 4 niveles de seguridad para evitar que los scripts puedan ser lanzados por cualquier usuario y puedan dañar el equipo.

Restricted → No se permite la ejecución del script.

AllSigned → Todos los scripts deben estar autenticados

RemoteSigned → Solo deberán estar autenticados aquellos que sea de forma remota.

Unrestricted → No tiene seguridad.

Get-ExecutionPolicy → Comprobamos que nivel de seguridad tenemos en nuestro equipo.

Set-ExecutionPolicy <nivel> → Cambiaríamos el nivel de seguridad.

Comentarios → Utilizamos el carácter '#'.

Variables → Se definen con el símbolo '\$'

Constantes → Son variables cuyo valor no cambia.

Las estructuras de control → Permiten modificar el flujo de ejecución de las instrucciones de un programa.

Snippets → Estructuras de código listas para ser utilizadas.

Estructuras condicionales → Ejecutan un grupo de sentencias en función del valor de una condición.

- if
- if-else
- switch

Estructuras repetitivas → Nos permiten repetir un bloque de instrucciones.

- while
- do-while
- for
- foreach

Funciones → Conjunto de instrucciones a las que le damos un nombre.

Ejemplo:

```
#Conexión con el servidor
#Definición de Funciones
Function Conectividad ($datos) {
    foreach ($i in $datos) {
        $respuesta=Test-Connection $i -count 1 -Quiet
        if ($respuesta -eq "true") {
            Write-Host "$i Conexión establecida"
        }
        else {
            Write-Host "$i Error de Conexión"
        }
    }
}
#Inicio del programa
Clear-Host
Write-Host "Conectividad"
$datos=Get-Content C:\Users\ferdu\Desktop\servidores.txt
Conectividad $datos
```

COMANDOS POWERSHELL

Principales Verbos

- **Get** → Obtiene información
- **Set** → Cambia una propiedad
- **Remove** → Elimina un objeto
- **New** → Crea un objeto

Principales Comandos

Get-Host → Ver la versión de Powershell

Update-Help → Actualizamos la ayuda

Get-Command -verb / -noun <verbo o nombre a buscar> → Nos devuelve todos los comandos que contengan ese nombre o verbo. Si lo que vamos a buscar lo ponemos entre asteriscos, añade búsquedas que contengan esa palabra.

Get-Help <nombre_comando> → Ayuda estándar de un cmdlet.

Get-Help <nombre_comando> -examples → Ayuda con ejemplos.

Get-Help <nombre_comando> -detailed → Ayuda detallada de un cmdlet.

Get-Help <nombre_comando> -full → Ayuda completa de un cmdlet.

New-LocalUser <nuevo_usuario> -Description “descripción” -NoPassword → Crea un nuevo usuario en el sistema, pero sin contraseña

Get-LocalUser → Vemos los usuarios locales de nuestro sistema

Remove-LocalUser <usuario> → Borramos al usuario local.

New-Item <nombre_directorio> -ItemType Directory → Creamos un directorio

New-Item .\<directorio>\<nombre_archivo> → Creamos un archivo en un directorio

Remove-Item <directorio> -recurse → Elimina un directorio

Move-Item <nombre_archivo> <nombre_directorio> → Mueve un elemento de un directorio a otro

Rename-Item <nombre_fichero o nombre_directorio> <nuevo_nombre> → Cambiamos el nombre del fichero o directorios